IDG2100 Full-stack Web Development: exam description

AI Literacy and Awareness Game

This is a draft of the final exam. If you find a typo or feel that some parts need clarification, contact the lecturers so we can edit the document. This document will not be edited after Monday, May 12. The last version of the document will be reuploaded in the same place it was posted originally.

This document contains the description of the final exam for IDG2100 Spring 2025, which will be done in groups of 3. The project must be delivered in this GitHub repository as well as in Blackboard. The project will be orally presented, and students will receive an individual grade based on their performance.

Notice this is an extension of the previous compulsory assignments (obligs). Therefore, if you decide to reuse code from those assignments, ensure it is properly cited.

Important deadlines are posted on Blackboard.

08/05/2025

Table of contents

Context *3*

Project exam *4*

Goal **4**

AI Literacy and Awareness Game – full-stack web app **4**

New requirements 5

Summary, System Flow and Functionalities **6**

Overview 6

User Roles and Interfaces 6

Functionalities 7

Grading **8**

Delivery **8**

Useful resources *9*

Context

This document contains the description of the final group exam.

This is a group assignment involving three students. Groups were created in the last compulsory assignment (oblig 4). You are permitted to utilise or integrate code from your previous mandatory assignments (obligs) if required. However, ensure that the same code is not utilised by members of other groups. Always cite the code clearly in the comments, identifying the student who created it and the "oblig" in which it was originally delivered.

This assignment is divided into two main parts: a full-stack project and an on-campus oral presentation (oral exam). You must submit your code via both GitHub and Blackboard (please refer to the *Delivery* section for specific instructions). The details regarding the dates and requirements for the project delivery and the physical oral presentation will be posted on Blackboard.

Although you may utilise snippets of code from tutorials and official documentation, you must clearly acknowledge the sources in the comments of your code. Plagiarism or cheating will be deemed to have taken place if the submitted code shows substantial similarities to other students' assignments or projects found online. In such cases, the matter will be reported to the NTNU appeals committee for further examination. If you have any doubts regarding the use of materials for your project, please reach out to the instructor for clarification.

Project exam

Goal

This exam aims to prove and demonstrate that you have achieved the learning outcomes, knowledge, skills and general competence described in the course description.

It is especially important that you show that:

- You understand the `MERN` technology stack and can build a full-stack modern web application using it,
- You can design a REST API following best practices for API design and Express.js server architecture,
- You can successfully document your projects using modern tools such as Swagger and Storybook,
- You can provide basic security to both the back-end and the front-end,
- You can use modern developer tools such as task runners, bundlers, etc.

AI Literacy and Awareness Game – full-stack web app

Great news! The NTNU Department of Design has seen all your hard work from previous assignments and they are now convinced they want to further develop the AI Literacy tool and invite researchers and educators to use it.

This final project incorporates elements from preceding mandatory assignments, complemented by additional features outlined below as well as new technical requirements. Consequently, all previously assigned tasks must be completed fully since they will also be part of this evaluation.

The system will now support three different types of users:

- **Visitors (non-authenticated users)**: Visitors will have access to the site landing page, quiz-list and researcher-list pages, and can take quizzes.

- **Researchers (authenticated users, also referred as educators)**: Researchers can create, manage, and view analytics info for their quizzes and access their quiz data.

- **Admins (authenticated users)**: Admins are responsible for platform administration, including managing users (researchers and admins) and overseeing the platform.

The landing page of the application will provide non-authenticated users with a comprehensive overview of the platform's features. Additionally, it will highlight the top quizzes (quizzes with more answers) and provide an option for users to register. Don't forget that users usually forget their passwords, and this is also a functionality that belongs to the registration process.

Researchers will have the option to create and manage quizzes, view user performance, and access demographic insights. Admins will have access to all common administrative functionalities found in other platforms, accessible via the web user interface.

All authenticated users will have the capability to generate reports and access detailed analytics about their quizzes. The remaining functionalities can be implemented across multiple pages, and it's essential to discuss them within the team, taking into account user experience considerations.

New requirements

- Authorisation and Authentication: Ensure both the front-end and back-end are secured, relying on JSON Web Tokens (JWT) and HttpOnly cookies to implement token-based authentication (with both access and refresh tokens).
    - Therefore, you don't need to use *localStorage*.Don't forget that when you reload the page, Researchers and Admins must remain authenticated if they logged in before refreshing the page.
- Support all user roles and their respective functionalities.
- Documentation Updates: Revise the documentation from Oblig 2 to reflect the current project status.
- API Documentation: Create comprehensive API documentation using Swagger.
- React components documentation: Document the most important React components using StoryBook
- Implement all the functionalities described in all the previous obligs and the previous section:
    - Have login and register pages, and a button that alternates between login and logout (depending on the state), and display a welcome message next to the logout button (e.g., "Hello, <username>."); only show the login/logout buttons if a navbar is visible (not to anonymous users while taking a quiz).
    - Have an admin dashboard with a) a list of requests for educator profiles and buttons to approve such requests, b) platform usage analytics, incl. the daily number of unique visitors for the week, total and last-week number of created and published quizzes, and platform load info (per-hour breakdown of numbers of requests to the backend), and c) functionality to search for, view, and ban user accounts.

    - Have email-based verification for new profiles: an email is sent after profile creation, and the user needs to click the link in it to verify their profile; the link is only valid for 15min and for that specific profile.
    - Have an educator dashboard with a) the list of their quizzes, b) the list of their uploads (files), c) functionality to create quizzes and (un)publish them.
    - Quizzes in the educator dashboard list should be clickable, leading to individual-quiz pages showing quiz status (draft, published, unavailable) and participation info (# of responses, # of started, but unfinished quizzes, and # of fully finished quizzes), and functionality to edit the quiz.
    - The system should allow for both at-home and in-museum quiz display, with the UI scaling well for mobile devices and wall-sized displays.
    - All the routes must be properly protected (back-end and front-end) and the routing in the front-end must consider all the scenarios (page not found, unauthorised, not authenticated) and their respective redirections. Redirections and user experience are important.
    - The front-end should also account for network delays and errors, displaying appropriate messages to the user (e.g., showing a loading animation when fetching data and disabling 'submit' buttons when uploading data).
- Don't forget to include all the necessary information to install the platform in any environment (e.g., lecturer's computer) together with all the seed functionality to populate the database (Readme.md file and scripts in package.json).

Summary, System Flow and Functionalities

This section contains a summary of the main functionalities described in the previous compulsory assignments. Please, do not solely rely on this summary. Read the functionalities from Oblig 2 and Oblig 3 for a more detailed description and use the overview context to ensure your final application fulfils the main requirements of the platform. In addition to that, some new functionalities are listed here.

Overview

The AI Literacy App aims to enhance AI literacy among users by providing interactive quizzes that help differentiate between AI-generated and human-generated content. The system supports three types of users: Visitors, Researchers, and Admins, each with specific roles and functionalities.

User Roles and Interfaces

1. **Visitors (Non-authenticated Users)**

    - Access the landing page.
    - Take quizzes to identify AI-generated content (quizzes must be redirected to the quiz page after a period of inactivity).
    - View top quizzes and general information about the platform.
    - Visitors can input their email address after playing the quiz (results) and their final results and explanations will be send to the user by email.

2. **Researchers (Authenticated Users)**

    - Create, manage, and analyse quizzes. All CRUD operations.
    - Access detailed analytics and reports.
    - Researchers can only see their resources.

3. **Admins (Authenticated Users)**

    - Manage user roles and oversee the platform.
    - Handle administrative functionalities, including user management.

<u>Functionalities</u>

1. **Quiz Interaction Flow**

    - Front Page: Users encounter a front page for a quiz, enticing them to take it.
    - Quiz Initiation: Users start a quiz and answer questions one by one.
        - Questions may involve identifying AI-generated content from images, text, videos, or music.
        - Users can navigate back and forth among questions, but answers cannot be changed after submission.
        - Feedback and explanations are provided immediately after each question or at the results page depending on the back-end configuration.
    - Demographics Questionnaire: After completing the quiz, users fill out a demographic questionnaire (age group, gender, familiarity with AI).
    - Results Page: Users see their performance, including the number of correct answers and comparison with other users.
    - Users can send the results of their quizzes with the feedback by email.

2. **Admin Panel**

    - Administrators: Manage user roles, oversee the platform, and handle high-level settings. This is, all CRUD operations to create and administrate researchers and users.

- Researchers: Create and manage quizzes, view analytics, and access data. All CRUD operations.

3. **Front-End App**

    - Interactive Quizzes: Users can take a quiz and guess whether the content is AI-generated or not.
    - Feedback and Explanations: Feedback with explanations on how to identify AI-generated content (immediate feedback vs feedback display at the end of the quiz, depending on backend configuration).
    - Score Tracking: Display scores, averages, and deviations to show how users compare to others.
    - Mixed Content Challenges: Combine different types of content for more complex challenges.

4. **Technical Requirements**

    - Frontend: Built using React.
    - Backend: Node.js and Express.js.
    - Database: MongoDB and Mongoose.
    - Secure Authorisation and Authentication: Using JSON Web Tokens (JWT) and HttpOnly cookies.
    - API Documentation: Comprehensive documentation using Swagger.
    - React Components Documentation: Document important components using StoryBook.
    - Email Validation: Allow users to register and validate their email.

5. **Routing and Protection**

    - Protected Routes: Ensure all routes are properly protected (back-end and front-end).
    - Routing Scenarios: Handle scenarios like page not found, unauthorized access, and unauthenticated access with appropriate redirections.

Grading

The project is done in groups (created in Oblig 4), and the source code is delivered in Blackboard and GitHub classroom and presented orally. All the members of the group must participate in the oral presentation physically on campus during the assigned time slot. Students will be graded individually based on their performance.

Delivery

This assignment must be delivered in two different places: **GitHub classroom** and **Inspera**.

- To deliver the assignment in GitHub Classroom, you only need to push your changes and commits to your Git repository. Make sure the final version of your code is merged into the `main` branch.
- Before delivering the assignment in inspera, make sure your project has all the files it needs. Delete any file or info not needed (this is `.git/` folder, `node_modules/`, etc.). Zip the project and upload the file to Inspera.

Useful resources

You can find additional resources in

Front-end

- React Router 7: Private Routes (alias Protected Routes)
- Creating Protected Routes With React Router V6 – Please, make sure you use Router V7 instead.
- (video) React Protected Routes | Role-Based Authorization | React Router v6 – Please, make sure you use Router V7 instead.
- (Video) The New Way To Create Protected Routes With React Router V6 – Please, make sure you use Router V7 instead.

Back-end

- Node.js + MongoDB API - JWT Authentication with Refresh Tokens
- React 18 Authentication with Node.js JWT API
- React + Axios - Add Bearer Token Authorization Header to HTTP Request